

TurtleBot3 在 Gazebo 中跑通 SLAM 建图与自主导航 (ROS1 Noetic 实战 + 踩坑记录)

> 项目: [xzy-project](../README.md) · 环境: Ubuntu 20.04 + ROS1 Noetic + Gazebo 11
· 车型: waffle_pi
> 状态: 已跑通 (2026-08-09) · 配套视频清单见 VIDEO_CHECKLIST.md

1. 背景

本项目的技术主线是"先车后船": 把自主无人系统的四层关键技术——**感知 → 规划 → 控制 → 协同**——先在地面小车 (TurtleBot3 仿真) 上跑通, 再迁移到水面/水下无人系统。算法与载体无关, 变的只是模型、动力学和传感器。

本博客记录的是这条主线上第一个**完整闭环**: 在 Gazebo 仿真里用 Gmapping 实时建图, 保存地图, 再加载地图用 AMCL 定位 + move_base 导航, 让小车自己规划路径走到目标点。"感知 → 定位 → 规划 → 控制"整条链路第一次串起来。

2. 环境

项	值
系统	Ubuntu 20.04.6 LTS
ROS	ROS1 Noetic (desktop)
仿真	Gazebo 11.15
车型	`waffle_pi` (360° 激光 + RGB 相机)
关键包	`turtlebot3` / `turtlebot3-simulations` / `navigation` / `gmapping`

```
```bash
sudo apt update
sudo apt install -y ros-noetic-turtlebot3 ros-noetic-turtlebot3-simulations \
 ros-noetic-navigation ros-noetic-gmapping
```
```

3. 核心概念速览 (一句话版)

- **SLAM (Gmapping)**: 激光 + 里程计实时构建栅格地图 (occupancy grid), 同时修正车自身的位置。它就是"边开车边画地图"。
- **定位 (AMCL)**: 给车一张已知地图, 用粒子滤波估计车在地图中的位置和朝向。粒子云收敛成一团 = 定位成功。
- **规划与控制 (move_base)**: 全局规划器找一条从当前位置到目标点的路径, 局部规划器 (DWA) 实时避障, 最终输出 `/cmd\_vel` 速度指令。
- **TF 坐标系链**: `map → odom → base\_footprint → base\_link → base\_scan`。其中 `map→odom` 由 AMCL 发布, `odom→base\_footprint` 由里程计发布, `base\_footprint` 往下的关节/传感器变换由 URDF 和 robot_state_publisher 发布。

4. 建图实操

开三个终端, 按顺序执行 (每个新终端都要先设车型环境变量):

> 以下指令是项目自定义 launch (封装官方 turtlebot3 包), 对照表见 [launch/README.md](../launch/README.md)。

****终端 1 — 仿真世界****

```
```bash
```

```
export TURTLEBOT3_MODEL=waffle_pi
roslaunch ~/xzy-project/launch/simulation_world.launch
```
```

****终端 2 — SLAM 建图（自动弹出 RViz）****

```
```bash
export TURTLEBOT3_MODEL=waffle_pi
roslaunch ~/xzy-project/launch/mapping.launch
```
```

****终端 3 — 键盘遥控****

```
```bash
export TURTLEBOT3_MODEL=waffle_pi
roslaunch ~/xzy-project/launch/teleop_keyboard.launch
```
```

`w` 前进、`a/d` 转向、`s` 停车，让车在房间里慢慢绕一圈，RViz 里的栅格地图会随激光扫描一点一点"长"出来。转完保存地图：

```
```bash
mkdir -p ~/xzy-project/maps
roslaunch ~/xzy-project/launch/save_map.launch
```
```

生成 `map.pgm`（栅格图像）+ `map.yaml`（地图元数据）。

5. 导航实操

****终端 1 保持 Gazebo 开着**，新开终端：**

```
```bash
export TURTLEBOT3_MODEL=waffle_pi
roslaunch ~/xzy-project/launch/navigation.launch
map_file:=$HOME/xzy-project/maps/map.yaml
```
```

RViz 弹出后按顺序做两件事：

1. ****设初始位姿****：点工具栏 `2D Pose Estimate`，在地图上车实际所在的位置（**建图出发点**，见坑 3）按住并拖出车头朝向，松手。
2. ****给目标点****：点 `2D Nav Goal`，在地图另一头点一下，拖出目标朝向，松手。

正常现象：红色粒子云先收敛成一团 → 出现绿色全局路径 → 小车沿路径行驶、遇障碍绕开 → 到达自动停。

6. 踩坑记录（本篇重点）

坑 1：松开键盘，小车一直不停

这不是"没阻力"。Gazebo 的差速驱动插件只认 `/cmd\_vel` 上最新一条速度指令，而 teleop 节点只有**按下键的瞬间**才发一条速度；松键不补发零速度，于是小车"巡航"在最后一条指令上。真车固件有看门狗（几百毫秒收不到新指令自动停），仿真里没有这个机制。

****结论****：想停就按 `s`（发零速度）；退出遥控前先按 `s`，否则最后一条非零指令会一直留在话题上。

验证：`rostopic echo -n1 /cmd\_vel`，松键后再看，话题上还是最后那条非零速度。

坑 2: map_server 启动即崩 (exit 255)

报错:

```
ERROR: failed to open image file "/home/xzy/map.pgm": Couldn't open
/home/xzy/map.pgm
[map_server-2] process has died [pid ..., exit code 255]
```

原因: `map.yaml` 里的 `image:` 是**绝对路径**。把 `map.pgm` 从家目录搬进 `xzy-project/maps/` 时只搬了文件、没改 yaml 里的路径, map_server 按旧路径找不到图片, 直接退出。

结论: `image:` 路径必须始终指向 `map.pgm` 的实际位置, 两个文件必须一起搬。搬完要检查:

```
bash
cat ~/xzy-project/maps/map.yaml # 应显示 image:
/home/xzy/xzy-project/maps/map.pgm
```

坑 3: RViz 里车在地图中心, Gazebo 里车却在出发点

这是最隐蔽的一个。`turtlebot3\_world.launch` 里车的生成点是 $(-2.0, -0.5)$, 而导航启动时 AMCL 的默认初始位姿是 $(0, 0, 0)$ ——所以 RViz 里车被画在了地图中心附近, 和真车差了约 2.5 米。激光扫出来的墙轮廓 (车周围固定的一圈) 自然和地图的墙对不上。

关键认知: **RViz 里车的位置不是从 Gazebo 读来的, 而是 AMCL 认为的位置。** 初始位姿给错, 后面全错。

结论: `2D Pose Estimate` 要设在地图上"建图出发点"的位置 (约地图中心往左 2m、往下 0.5m), 箭头朝车头方向。最稳妥的办法是边拖边看激光轮廓, 和地图墙线重合了就松手。

坑 4: 不设初始位姿, 导航完全起不来

AMCL 不收到初始位姿 (或初始位姿参数全为 0 时) 不会发布 `map → odom` 变换, costmap 一直等不到 `map` 坐标系, 整条链路卡死。这也是为什么"2D Pose Estimate 这一步不能省"。

7. 结果验证清单

- [x] `rostopic echo -n1 /scan` 有激光数据
- [x] tf 树存在 `map → odom → base\_footprint → ...` 完整链路 (`roslaunch tf\_monitor map odom`)
- [x] 粒子云从散开收敛聚成一团
- [x] 激光轮廓与地图墙线重合
- [x] 发目标后出现全局路径 → 局部路径 → 小车到达自动停

8. 与手写 SLAM 的联系 (m1)

本仓库 m1 模块是手写的 2D 栅格 SLAM: log-odds 栅格更新 + 相关扫描匹配。Gmapping 可以看成它的"工业版"——同样是栅格地图 + 扫描匹配, 只是加了粒子滤波和更完整的工程实现。先看懂 m1 的最小闭环, 再操作 Gmapping 时概念完全对得上。后面计划把 m1 的手写 SLAM 包成 ROS 节点, 和 Gmapping 做同场景对比。

9. 下一步

1. 用 Gazebo Building Editor 搭一个自己的环境，替换官方 `turtlebot3\_world`，重跑建图 + 导航（从"跑官方 demo"变成"自主搭建"）
2. 迁移到 ROS2 Nav2（集群阶段的主场）
3. 多车编队：把 m4 领航-跟随接进 Gazebo，开三台 TurtleBot3
4. 每完成一环录 30 秒视频存档（见 VIDEO_CHECKLIST.md）

10. 参考资料

- ROBOTIS e-Manual: TurtleBot3 仿真/建图/导航
- ROS Wiki: gmapping / amcl / move_base / map_server
- [ENV_SETUP.md](../ENV_SETUP.md)（环境与主线命令）